

Perceptrons and the Perceptron Learning Algorithm

Weighted threshold units, and how they learn

Fundamentals of Deep Learning · Class 3

29 July

Outline

From Threshold Logic to the Perceptron

The Learning Problem

The Perceptron Learning Algorithm

Convergence and Limits

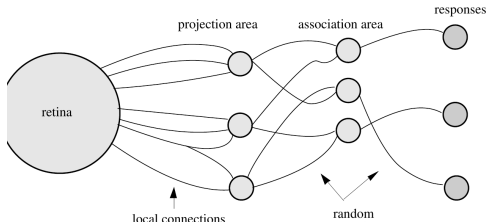
From Threshold Logic to the Perceptron

Where Class 2 left us

- A McCulloch–Pitts unit fires when $x_1 + \dots + x_n \geq \theta$ — inputs are summed with equal weight
- Geometrically it is a separating plane, but the planes are always parallel: only the offset changes
- Almost every Boolean function is out of reach — by $n = 5$ barely 0.002% are linearly separable

Give each input its own weight and add a bias: the plane can now take any orientation and offset. This is the perceptron.

A little history: Rosenblatt's perceptron

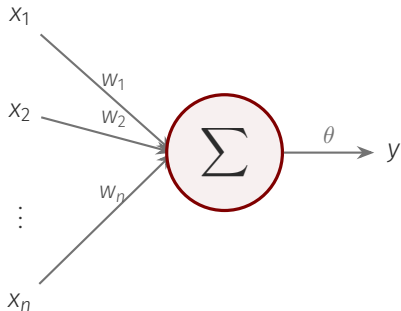


Rojas (1996), Fig. 3.1: the classical perceptron [after Rosenblatt 1958].

- Rosenblatt (1958) wired a **retina** of pixels to weighted threshold units — built to **learn** to recognise patterns
- Its only formal difference from a McCulloch–Pitts unit is the presence of **weights**
- It drew great excitement — and, later, a famous controversy over its limits

Rojas (1996), §3.1

The perceptron: a weighted threshold unit



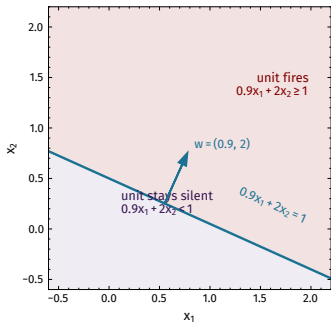
$$y = \begin{cases} 1 & \text{if } \sum_i w_i x_i \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

- Each input carries a real **weight** w_i
- Rosenblatt (1958); weights make the plane **freely orientable**

M-P fixed unit weights

PERCEPTRON **learnable** real weights

Weights tilt the separating line

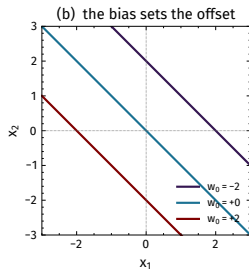
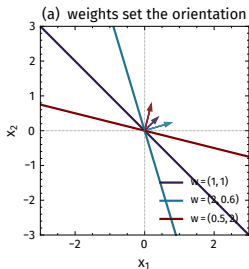


A perceptron with weights $(0.9, 2)$ and threshold 1 tests

$$0.9x_1 + 2x_2 \geq 1.$$

- The equality is the **boundary**; the two inequalities name the two **regions**
- w is **normal** to the line and points into the firing region
- The slope is now **arbitrary**

What the weights and the bias each control

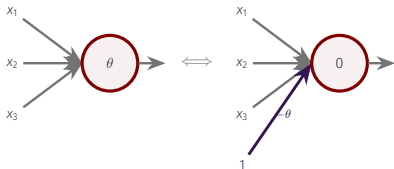


(a) three weight vectors with the same bias: the boundary $w \cdot x = 0$ takes a different *orientation* in each case (arrows show w , always normal to its line). (b) one weight vector with three biases: the boundary keeps its orientation and simply *shifts*.

M-P parallel planes only

PERCEPTRON any orientation and offset

The bias trick



Both units compute the same function:

$$\sum_i w_i x_i \geq \theta \iff \sum_i w_i x_i + (-\theta) \cdot 1 \geq 0.$$

- An input clamped to 1 with weight $-\theta$; the unit now compares against **zero**
- That weight is the **bias** — **learned like any other**, so one rule updates weights and threshold together

Rojas (1996), §3.1.1

M-P threshold θ fixed by the designer

PERCEPTRON threshold becomes a **learnable** bias weight

Extended input space

With the bias folded in, every input becomes an **extended vector** and the test is homogeneous:

$$\tilde{\mathbf{x}} = (1, x_1, \dots, x_n), \quad \tilde{\mathbf{w}} = (-\theta, w_1, \dots, w_n), \quad \tilde{\mathbf{w}} \cdot \tilde{\mathbf{x}} \geq 0.$$

- One vector holds everything — no separate threshold
- The separating plane now passes through the **origin** of $(n+1)$ -space
- Data split into P (target 1) and N (target 0); seek $\mathbf{w}$ with $\mathbf{w} \cdot \mathbf{x} > 0$ on P , < 0 on N

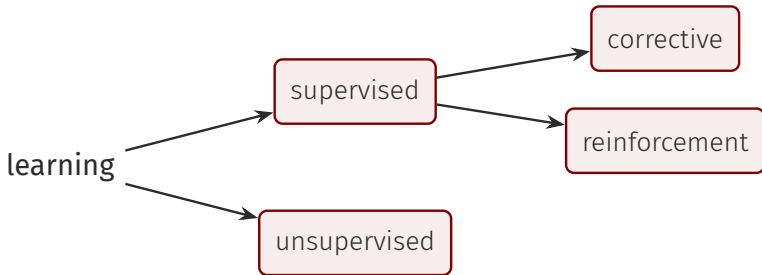
Rojas (1996), §4.1.2

M-P hand-designed weights

PERCEPTRON weights to be **found** from labelled data

The Learning Problem

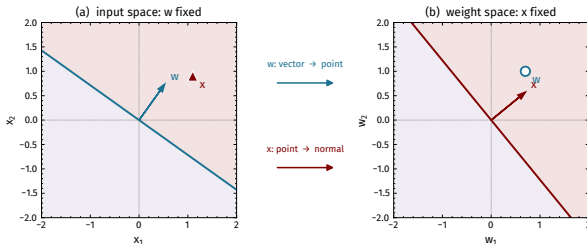
What does it mean to 'learn'?



- **Supervised**: each input comes with a target; **unsupervised**: the network finds structure on its own
- Supervised splits into **corrective** (fix the error directly) and **reinforcement**
- The **perceptron** rule is supervised with error correction

Rojas (1996), §4.1.1

Two dual viewpoints



$w \cdot x = 0$ is symmetric in w and x . (a) fix w and the *inputs* split into half-planes. (b) fix one input x and the *weights* split the same way — x becomes the normal, each candidate w a point.

Learning is a search in **weight space** for a point on the correct side of every data plane.

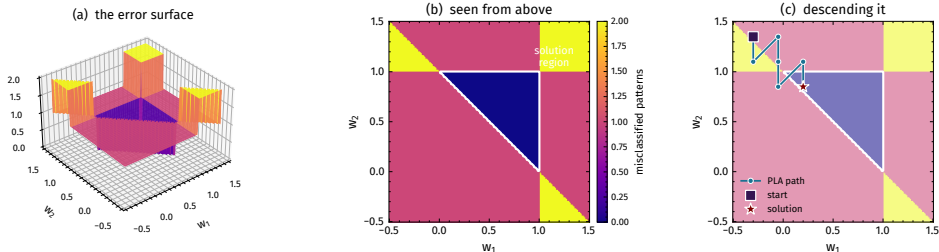
Learning as a search in weight space

- A network is a **parametric system**: fix the architecture, and the weights are free parameters
- “Learning” means **searching** weight space for values that classify the training data correctly
- Every input vector $\mathbf{x}$ imposes one constraint — it carves weight space into the weights that get it right and those that do not
- The solutions form the **intersection** of all those half-spaces Rojas (1996), §3.3.2

M-P weights fixed by the designer

PERCEPTRON weights **searched** to fit the data

The error surface



A perceptron with fixed threshold $\theta = 1$ learning **AND**; the error is the number of the four patterns it gets wrong, as (w_1, w_2) vary. (a) the surface as a landscape — flat terraces with sharp steps, since the error is a count. (b) the same surface from above; the white outline bounds the triangular **solution region** where the error is zero. (c) the path the algorithm takes across it. The three panels are the *same* surface: a landscape, a map, and a route across it.

after Rojas (1996), Fig. 4.5

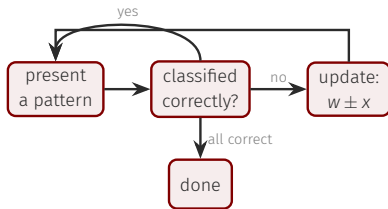
Absolute linear separability

- The PLA needs a **strict** separation: every point of P on the *open* positive side, every point of N on the open negative side
- If a separating plane exists, one can always be found with a **margin** — no data point lying exactly on it
- This **absolute** separability is what makes the convergence proof work Rojas (1996), §4.1.3

“Separable” is not quite enough — we need a little room on each side. Then learning is guaranteed to finish.

The Perceptron Learning Algorithm

Learning as error correction



- Start from a **random** weight vector; present training examples one at a time
- Classified **correctly** → change nothing
- **Wrong** → nudge the weights toward the right decision, then carry on
- Each step uses **one example** and the **current** weights only — local, online, and strikingly simple

The algorithm

Sets P and N of extended vectors. Seek $\mathbf{w}$ with $\mathbf{w} \cdot \mathbf{x} > 0$ on P and $\mathbf{w} \cdot \mathbf{x} \leq 0$ on N (Rojas, 1996, §4.2):

start: random $\mathbf{w}_0$, $t := 0$

test: pick $\mathbf{x} \in P \cup N$ at random

if correctly classified: go to **test**

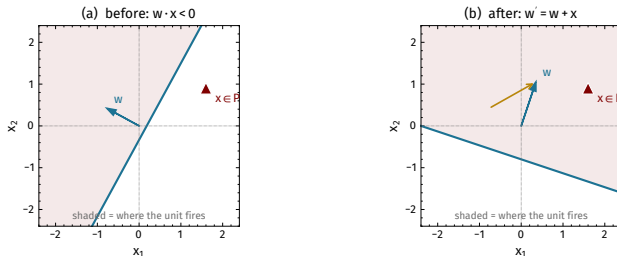
if $\mathbf{x} \in P$ and $\mathbf{w}_t \cdot \mathbf{x} \leq 0$: go to **add**

if $\mathbf{x} \in N$ and $\mathbf{w}_t \cdot \mathbf{x} \geq 0$: go to **subtract**

add: $\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{x}$, $t := t + 1$; go to **test**

subtract: $\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{x}$, $t := t + 1$; go to **test**

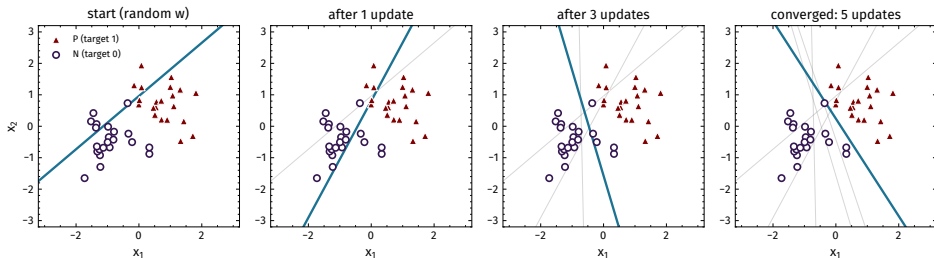
Why the update works



A point $x \in P$ the current w gets wrong: it lies outside the shaded firing half-plane. (a) before, $w \cdot x < 0$. (b) after $w' = w + x$, the weight has rotated toward x (amber arrow) and the half-plane now covers it.

$w' \cdot x = w \cdot x + \|x\|^2$ — the dot product **always increases**.

The algorithm running

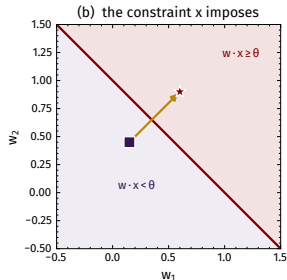
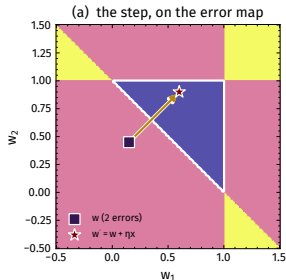


Two separable clouds, P (triangles) and N (circles). The teal line is the current decision boundary; faint grey lines are the boundaries it held earlier. Each panel is a later moment in the same run — the boundary swings across the data and stops as soon as every point is on its correct side.

M-P no update rule — weights set by hand

PERCEPTRON each mistake rotates w the right way

The update, seen in weight space



The same AND problem as before. (a) one correction moves w (square) to w' (star) — straight off a high-error terrace and into the solution region. (b) the same step against the constraint that the offending pattern x imposes: it demands $w \cdot x \geq \theta$, and the update carries w across that line. Each pattern contributes one such constraint; a solution satisfies them all at once.

after Rojas (1996), Fig. 4.11

Fixing the error in one step

The plain rule nudges; we can instead **fully correct** a mistake. For $\mathbf{x} \in P$ with $\mathbf{w} \cdot \mathbf{x} = -\delta \leq 0$, set (Rojas, 1996, §4.2.3):

$$\mathbf{w}' = \mathbf{w} + \frac{\delta + \varepsilon}{\|\mathbf{x}\|^2} \mathbf{x} \implies \mathbf{w}' \cdot \mathbf{x} = \varepsilon > 0.$$

- One update moves $\mathbf{x}$ just past the boundary, by a margin ε
- Small ε avoids overshooting into a worse region
- This **corrective** variant learns faster than plain add/subtract

M-P no notion of correction

PERCEPTRON each step **eliminates** one error outright

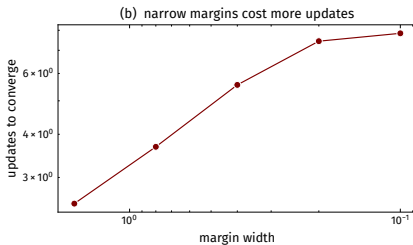
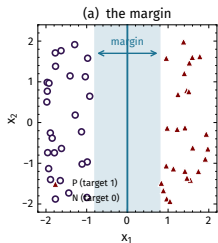
Convergence and Limits

The perceptron convergence theorem

If P and N are **linearly separable**, the perceptron learning algorithm makes only a **finite** number of weight updates — it reaches a separating $\mathbf{w}$ in finite time.

- Idea of the proof: if a solution $\mathbf{w}^*$ exists, every update increases the alignment $\mathbf{w} \cdot \mathbf{w}^*$ faster than it grows $\|\mathbf{w}\|$
- Bounding those two quantities forces the number of updates to be finite Rojas (1996), §4.2–4.3
- A guarantee of **termination**, not of speed

How long can it take?

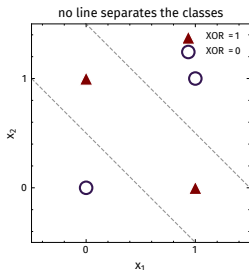


(a) the *margin* is the empty corridor between the classes. (b) average updates as that corridor narrows (log-log, margin shrinking to the right): the cost climbs with the inverse squared margin.

M-P no learning — nothing to time

PERCEPTRON finite, but slow when the margin is thin

The limit: XOR and non-separable data



- A single perceptron computes exactly the **linearly separable** functions — any orientation, but still one line
- **XOR** is not separable, so the PLA **never terminates** on it
- On non-separable data the **pocket algorithm** keeps the best weights seen so far

Rojas, §4.2.4

One layer is still a linear classifier. XOR needs a **hidden layer** — the shallow network (Classes 5–7).

Takeaways

1. A **perceptron** is a threshold unit with **learnable weights** and a **bias** — its boundary is a freely oriented hyperplane
2. The bias trick makes the test homogeneous: $\tilde{\mathbf{w}} \cdot \tilde{\mathbf{x}} \geq 0$
3. The **PLA** corrects only on mistakes: add $\mathbf{x}$ for a missed positive, subtract for a missed negative
4. It **provably converges** on separable data — but slowly when the margin is thin, and never on XOR

Class 5: stack weighted units into layers — the shallow neural network.

References

- R. Rojas. *Neural Networks: A Systematic Introduction*. Springer, 1996. Chapters 3–4 — weighted networks, input/weight-space duality, the learning algorithm and its convergence.
- F. Rosenblatt (1958). “The perceptron: a probabilistic model for information storage and organization in the brain.” *Psychological Review*, 65(6), 386–408.
- Plots generated with `src/make_figures.py` (NumPy, Matplotlib, SciencePlots).

Questions?